Cos'è un programma?

Lezione 1 — 1IOT, SST-ICT-01 / SST-INF-01c — Introduzione alla programmazione

15/09/2026

1 Organizzazione del lavoro

Da questa lezione in poi useremo un **quaderno** a quadretti dedicato esclusivamente alle materie di questo corso, **separato dalle altre materie**: non un blocco unico che mescola programmazione, italiano, matematica e inglese insieme. Un'idea pratica per chi ha una sola materia tecnica su un solo quaderno: **dividerlo a metà**, usando un lato per una materia e girandolo per usare l'altro lato per l'altra.

Nota

Tenere gli appunti organizzati non è un dettaglio burocratico: una **materia tecnica** come questa si costruisce **un concetto sopra l'altro**. Se **manca un pezzo intermedio** nei tuoi appunti, il pezzo successivo non si capisce più.

2 Che cos'è un programma?

Proviamo a rispondere insieme, partendo da alcuni esempi che ***non*** sono la risposta:

- "Il computer che fa giocare a un **videogioco**" – descrive **un *effetto***, non cosa *è* un programma.
- "Una **sequenza alfanumerica**" – ci si avvicina, ma non basta.
- "Un'applicazione come **Scratch**" – anche qui, descrive uno **strumento**, non la definizione.

Un modo semplice per arrivare alla **definizione** è un esperimento pratico: dare a una persona una **sequenza di ordini** (*alzati*, *esci dalla postazione*, *salta*, *siediti*). Se ripetiamo la stessa sequenza, la persona rifà esattamente le stesse azioni.

Definizione

Un **programma** è una **lista di istruzioni, ordinata**.

La parola chiave è **ordinata**: se **cambiamo l'ordine** delle istruzioni, il programma **cambia** – e potrebbe non funzionare più come vogliamo. Se do l'ordine "siediti" prima di "alzati", il risultato non ha senso: il programma si è "rotto" per un problema di **sequenza**, non perché l'esecutore abbia sbagliato.

Esempio

Una **ricetta di cucina** è un programma: è una **lista di azioni** (gli ingredienti, i passaggi) che va seguita in un **ordine preciso**. Se si **sbaglia l'ordine** – ad esempio si assaggia la pasta **prima di cuocerla** – il risultato non è quello desiderato.

Non tutte le **liste** sono programmi in questo senso stretto: una **lista della spesa**, ad esempio, di solito funziona anche se **cambiamo l'ordine** in cui mettiamo gli oggetti nel carrello.

Nota

Il computer non sbaglia mai da solo. Se un programma non funziona, la causa è sempre un **errore umano** in fase di scrittura del programma – una **sequenza di istruzioni sbagliata**, non una "decisione" della macchina.

Esempio

Pensiamo a un **robot** che si muove per la stanza e a un certo punto trova un **gradino**: cade e **non si rialza** più. La colpa non è del robot che "ha sbagliato": è del **programmatore** che non gli ha spiegato cosa fare in quella situazione. Quando un programma non funziona, la domanda giusta non è "perché la macchina ha sbagliato?" ma "**chi ha scritto il programma**, e cosa ha dimenticato di dirgli?".

2.1 Anche l'intelligenza artificiale funziona per probabilità

Un dubbio comune: "ma un'**intelligenza artificiale** come ChatGPT non si programma da sola?". No: dietro c'è comunque un **modello statistico** che calcola qual è la risposta ***più probabile*** in base a quello che gli è stato scritto – non necessariamente quella ***vera***.

Esempio

Per capire cosa vuol dire "**modello statistico**" partiamo da un esempio più vicino a noi: in una **classe di 14 persone**, qual è la **probabilità** che tutti e 14 vengano **promossi** alla fine dell'anno? Non possiamo saperlo con certezza – siamo tutti diversi, con attitudini diverse – ma possiamo fare una **stima** basata su quello che è successo negli anni scorsi: generalmente, in una classe di 14 persone, circa 2 vengono **bocciate** e 12 proseguono. Questo non significa avere "**la verità in mano**": è solo una **probabilità** calcolata sui **dati passati**. Può succedere che vengano promossi tutti, così come può succedere che ne vengano bocciati di più – il modello statistico dà la **stima** più *plausibile*, non una **certezza**.

In pratica, un modello come ChatGPT valuta un gran numero di risposte possibili e restituisce quella con il **punteggio di probabilità** più alto – non ha "la verità in mano", calcola solo qual è la risposta **più plausibile** in base a quello che gli è stato scritto. Attenzione: l'**errore comune** è pensare che, parlando con ChatGPT, Claude o un'altra IA, si abbia davvero a che fare con "**qualcuno**" che risponde con certezza. Non è così: sono tutti **modelli informatici** che rispondono in funzione della **probabilità**.

Esempio

Un **caso concreto**: un problema con **Windows** che non si avviava correttamente veniva diagnosticato dall'IA come un problema di **driver** – ma la **causa reale** era tutt'altra. La macchina non ha "**mentito**": ha semplicemente calcolato la risposta **statisticamente più probabile** in base alle informazioni (**incomplete**) che le erano state fornite. Se le informazioni

date all'IA sono **incomplete o sbagliate**, anche la **risposta sarà sbagliata**, senza che sia necessariamente "colpa" di chi ha chiesto.

Attenzione

Anche se una risposta di un'intelligenza artificiale è corretta la maggior parte delle volte (anche al 99,9%), esiste sempre una **probabilità di errore residua**. Non bisogna **mai** copiare/incollare senza **verificare**: se le informazioni fornite all'IA sono incomplete o sbagliate, anche la risposta sarà sbagliata – e l'**errore** è sempre dietro l'angolo.

Nota

Anche queste dispense vengono scritte con l'**aiuto dell'IA**. Questo non significa che siano prive di **errori**: vanno sempre **rilette** con attenzione prima di essere pubblicate, perché un semplice **copia-incolla** senza verifica potrebbe introdurre imprecisioni. Se trovi un errore in una dispensa, segnalalo pure: verrà corretto.

3 Dal linguaggio ai numeri: perché serve il binario

Il problema successivo è pratico: i **computer** non capiscono le **parole**, capiscono solo **numeri**. Potremmo dare un **codice numerico** a ogni **istruzione** (ad esempio: 23 = "alzati", 42 = "esci", 12 = "siediti"), ma resta un problema di fondo – *come* rappresenta un **numero**, internamente, una macchina che non "vede" le cifre come le vediamo noi?

3.1 L'esperimento delle lampadine

Immaginiamo due persone su due **isole** diverse, senza telefono, che vogliono scambiarsi **informazioni a distanza** usando solo delle **lampadine** (**accese o spente**).



Figura 1: Con 2 lampadine per isola si possono segnalare 4 situazioni diverse (es. via libera / squalo / medusa / piranha), una per ogni combinazione di acceso/spento.

Con **1 lampadina** si possono comunicare 2 **informazioni diverse** (accesa / spenta) – ad esempio "**via libera**" oppure "attenzione, ci sono squali". Con **2 lampadine** le **combinazioni possibili** diventano 4: **entrambe spente** (via libera), **solo la prima accesa** (squalo), **solo la seconda accesa** (medusa), **entrambe accese** (piranha) – quattro situazioni diverse comunicate con solo due segnali acceso/spento. Con **3 lampadine** si arriva a 8 **combinazioni**, con **4** a 16.

Definizione

Regola generale: ogni lampadina aggiunta ***raddoppia*** il **numero di informazioni rappresentabili**, perché ogni lampadina ha 2 **stati possibili**, indipendenti dalle altre.

$$\text{numero di combinazioni} = 2^n$$

dove n è il **numero di lampadine**.

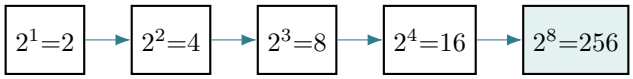


Figura 2: Ogni lampadina (bit) in più raddoppia le combinazioni rappresentabili: con 8 lampadine se ne ottengono già 256.

La **crescita** è molto più **rapida** di quanto sembri a prima vista: con **16 lampadine** si arriva già a $2^{16} = 65\,536$ **combinazioni diverse** – un numero enorme, ottenuto raddoppiando ripetutamente a partire da una **singola lampadina**.

3.2 Perché non una lampadina "a intensità variabile"?

Si potrebbe pensare di usare una lampadina che varia di **luminosità** con continuità, invece che solo **accesa/spenta**, per rappresentare più informazioni con un solo elemento. Il problema è la **precisione**: regolare e leggere con esattezza un livello di luce continuo è molto più **difficile** e soggetto a errori rispetto a un semplice **stato acceso/spento**.

Esempio

È la stessa ragione per cui un'istruzione come "alzati" o "siediti" è facile da eseguire in modo **affidabile**, mentre "alzati di **3 cm esatti**" richiederebbe sensori e una precisione che introduce margini di errore.

Il computer ragiona esattamente con "lampadine" di questo tipo: si chiamano **bit**, possono valere solo **0** o **1**, e ogni numero che conosciamo può essere rappresentato come una **combinazione** di bit accesi o spenti. Ad esempio, il numero 254 corrisponde a una precisa combinazione di **8 bit**.

Nota

Come si trasforma esattamente un numero (ad esempio 254) nella sua **rappresentazione in bit** sarà l'argomento della prossima lezione.

4 Cosa ricordare di questa lezione

- Un **programma** è una **lista di istruzioni ordinata**: l'**ordine conta** sempre.
- Se un programma non funziona, l'**errore è sempre umano**: il computer esegue esattamente quello che gli viene scritto.
- Anche un'intelligenza artificiale funziona per **probabilità**, non per certezza: va sempre **verificata**.

- I computer rappresentano tutto con **bit** (0 o 1), non perché sia comodo per noi, ma perché uno stato **acceso/spento** è affidabile e privo di ambiguità.
- Con n bit si possono rappresentare 2^n combinazioni diverse: è la base di tutta l'informatica che vedremo da qui in avanti.

Esempio

Da fare per la prossima lezione: prova a chiederti quante **lampadine** (bit) servirebbero per rappresentare tutte le **lettere dell'alfabeto** italiano. Non serve la risposta esatta: basta ragionarci con la formula 2^n.